OASIS: Repairing Stale Optimizer Statistics in the Feedback Nullspace

Qichu Tian[a,*], Heng Chen[a]

[a] *Xi'an Jiaotong University, Xi'an, China*

Abstract

Cost-based optimizers plan with column statistics that grow stale between refreshes, and query feedback exposes the resulting selectivity errors. Repairing a histogram from feedback is underdetermined: a few observed predicates constrain the marginal only where they fall and leave the rest free—a region we call the feedback nullspace. Classical feedback-consistency methods (STHoles, ISOMER, QuickSel-H) fill this region with an optimizer-agnostic default. The deployment question is therefore not how to match the observed feedback more tightly, but what to place in the nullspace.

We propose OASIS (Optimizer-Aware Statistics Imputation System), a statistics-layer middleware that imputes the nullspace with a completion learned against the optimizer's own error. A learned imputation layer produces a marginal prior from a composite optimizer-facing objective. A calibration layer projects that prior onto the feedback and routes among candidates using a deployment-visible signal.

A rank analysis explains when this helps: the gain is concentrated in the nullspace the feedback leaves and vanishes once dense feedback pins the marginal. On held-out drift, OASIS cuts future-predicate Q-error by 12.8% over ISOMER and also beats STHoles and QuickSel-H, with the gain largest where feedback is sparse—a 2.7× advantage at the sparsest budget on a real PostgreSQL planner. It also improves a composition family, repairs a FactorJoin kernel an uncalibrated prior would harm, and on a TPC-H check tracks fresh-statistics behavior without a table scan.

Keywords: Statistics correction, Cardinality estimation, Histogram

*Corresponding author

Email addresses: mizukiqwq@stu.xjtu.edu.cn (Qichu Tian), hengchen@xjtu.edu.cn (Heng Chen)

calibration, Cost-based optimizer, Query feedback, Underdetermined inverse problem

1. Introduction

Cost-based optimizers (CBOs) choose query plans from estimates of predicate selectivity, intermediate cardinality, and operator cost [1, 2]. In many analytical database systems these estimates rely on column-level statistics such as histograms, most-common-value lists, and density summaries [3, 4, 5, 6], refreshed periodically by `ANALYZE`-style procedures that read a full or sampled view of the table [7]. This maintenance strategy is simple and robust, but it leaves a persistent staleness gap: tables keep receiving inserts, deletes, and updates while the optimizer plans with the last statistics snapshot. As the gap widens, stale histograms induce systematic selectivity errors that propagate through the cost model into poor plans [8, 9, 10, 11].

A DBMS already obtains useful signal while executing ordinary queries. For a filter predicate, the optimizer's estimated selectivity and the executor's observed selectivity expose a local discrepancy between the stale histogram and the current data distribution. Feedback-driven self-tuning histograms turn such observations into statistics updates—STHoles, SASH, and ISOMER are the canonical examples [12, 13, 14, 15]—and ISOMER in particular makes the underlying principle explicit: among all marginals consistent with the observed feedback, choose the *maximum-entropy* one. This is a principled but optimizer-agnostic choice: the least-committed completion of the part of the marginal that feedback does not determine.

The observation that organizes this paper is structural. An equi-depth histogram with B buckets has $B - 1$ free interior quantile boundaries. A feedback window of K predicates constrains the cumulative distribution at the predicates' boundary values—at most one independent constraint per distinct boundary. When K is small relative to B, or when the observed predicates cluster, the feedback pins the marginal only along the span of those constraints and leaves the orthogonal complement—the *nullspace*—free. Any feedback-consistency method, ISOMER included, must *complete* the marginal in that nullspace, and the quality of an optimizer-facing statistic is governed by how well it does so on predicates that fall where feedback did not. Maximum entropy is one completion; it is not the completion that minimizes downstream optimizer error.

This reframing changes the deployment question from "how do we match the feedback better" to "what should we put in the nullspace." It also sets two boundaries that we make quantitative rather than rhetorical. First, a completion can only help where a nullspace exists: once feedback saturates the available degrees of freedom, the marginal is pinned and *every method applying the same finite projection* coincides. Second, a completion learned to reproduce the post-drift histogram—the obvious supervised target—does not help either, because reconstruction accuracy is not the same as optimizer relevance.

The resulting system is OASIS (Optimizer-Aware Statistics Imputation System), which imputes the feedback nullspace with a learned, optimizer-relevant completion. OASIS has three layers. A *conversion layer* maps native engine statistics to a canonical distribution view and back (Section 3.2). A *learned imputation layer* produces a marginal prior from a compact permutation-invariant model trained on a composite objective that scores the projected marginal by its held-out future-predicate error, a FactorJoin join-regret term, an in-window feedback-consistency residual, and a reconstruction regularizer (Section 3.3). A *calibration layer* reuses the classical feedback projection as a completion operator over any prior and adds a residual-gated router that selects between the learned completion and the maximum-entropy default from a deployment-visible signal (Section 3.4). This design yields four contributions:

- We formulate feedback-driven marginal repair as completion of an *underdetermined* system and give a rank characterization of the nullspace that predicts when a learned prior can help and when it must tie the maximum-entropy projection (Sections 2.3, 4).
- We train the nullspace completion on a composite optimizer-facing objective and show it beats both reconstruction-trained priors and classical self-tuning histograms wherever feedback under-determines the marginal; an ablation isolates the gain to the objective and shows in-loop projection is inessential (Sections 3, 6.1.3).
- We show the corrected marginal is safe to consume: it improves a six-estimator composition family by 20%–36%, rescues a FactorJoin kernel where the raw prior regresses below stale, and reduces PostgreSQL new plan deviations to 1.1%, while a residual-gated router gives the same safety from a deployment-visible signal (Section 6.2).
- We give a primarily optimizer-facing evaluation spanning single-column drift, structural and out-of-distribution generalization, the composition family

and FactorJoin kernel, a 12-configuration PostgreSQL planner-only study, a three-seed TPC-H runtime check, and sparse/noisy feedback diagnostics.

2. Background and Problem Formulation

2.1. Feedback-Based Statistics Maintenance

Because an `ANALYZE`-style refresh must read a full or sampled view of the table, refreshes are scheduled rather than continuous; between them the table keeps changing while the optimizer plans from the last snapshot, so a once-accurate histogram drifts from the current distribution—especially around recent values, range predicates, and localized updates.

Query execution already exposes this drift: for each filter predicate, the optimizer's estimated selectivity and the executor's observed selectivity reveal a local disagreement between the old statistics and the current data. Feedback-driven self-tuning histograms use such observations to repair statistics: STHoles splits and refines buckets from feedback [13]; ISOMER selects the maximum-entropy histogram among those consistent with the feedback and drops old constraints when drift makes the active set inconsistent [15]; QuickSel-H fits a query-driven selectivity model on the same no-rescan interface [16]. These methods all take query feedback as input and emit a corrected single-column statistic without touching the base data. Their key difference is how they complete the parts of the distribution that feedback does not determine. OASIS keeps the same interface but learns this completion against optimizer error instead of fixing it by maximum entropy or bucket splitting.

2.2. Statistics and Objective

We model a column histogram as an equi-depth quantile summary with B buckets. Its interior boundaries $v_1, \ldots, v_{B-1}$ correspond to fixed quantile levels $p_1, \ldots, p_{B-1}$ and induce a piecewise-linear CDF used to estimate range-predicate selectivity. Native DBMS statistics may also include most-common-value lists, density vectors, or null fractions; OASIS operates on a canonical one-dimensional CDF view and converts to and from native statistics through an engine-specific adapter (Section 3.2).

Let H_0 be the stale histogram from the last refresh and H^* the current post-drift distribution. For a predicate σ, the optimizer derives an estimate $\hat{s}_{H_0}(\sigma)$ from H_0, while the executor can later reveal an actual selectivity $s^*(\sigma)$. A feedback window $\mathcal{O} = \{o_1, \ldots, o_K\}$ records such observations; each

o_j contains a predicate type, a boundary, an estimated selectivity, and an actual selectivity. We measure error with Q-error,

$$\mathrm{QErr}(\hat{s}, s^*) = \max\left(\frac{\hat{s}}{s^*}, \frac{s^*}{\hat{s}}\right). \tag{1}$$

For one predicate, Q-error is a scalar. To compare two statistics objects, we average this scalar over future predicates. Let $\sigma \sim \mathcal{D}_{\mathrm{fut}}$ denote a future predicate drawn from the workload distribution, and let s^*_σ be its true selectivity under the current distribution H^*. The goal is to construct corrected statistics $\hat{H}$ that lower this average future-predicate error,

$$\mathbb{E}_{\sigma\sim\mathcal{D}_{\mathrm{fut}}}[\mathrm{QErr}(\mathrm{CDF}_{\hat{H}}(\sigma), s^*_\sigma)] < \mathbb{E}_{\sigma\sim\mathcal{D}_{\mathrm{fut}}}[\mathrm{QErr}(\mathrm{CDF}_{H_0}(\sigma), s^*_\sigma)]\,.$$

A deployable repair must derive $\hat{H}$ from H_0 and feedback only, avoid table rescans, run near planning time, and degrade gracefully when feedback is sparse.

2.3. Degrees of Freedom in Feedback-Based Repair

Feedback repair is underdetermined for an elementary reason. A feedback window reports the true mass on a handful of intervals; between and beyond those intervals, the data can be arranged in many ways and still reproduce every observation. The repaired marginal is thus *pinned* where feedback falls and *free* everywhere else, and that free part—the feedback nullspace (Figure 1)—is what any repair method must fill. The rest of this section makes this precise and measures how large the free part is.

To make this precise, we rewrite the feedback constraints in a representation that makes them *exactly linear*. Take the active feedback predicates together with the stale histogram grid; their boundary points—the grid points and the distinct feedback endpoints—partition the domain into C cells. We then represent the marginal by its vector of cell masses $m \in \Delta$. Since these C masses are nonnegative and sum to one, the simplex Δ has dimension $C - 1$.

In this *cell-mass representation*, each feedback observation constrains only the total mass on its predicate interval. Because every predicate endpoint is a cell boundary, that interval mass is an exact sum of cell masses (two such sums for a `BETWEEN`). The feedback window therefore imposes a system of linear equality constraints $Am = y$, with $A \in \mathbb{R}^{K'\times C}$. Let $r = \mathrm{rank}(A)$. The marginals consistent with the feedback form the affine slice $\{m \in \Delta : Am = y\}$,

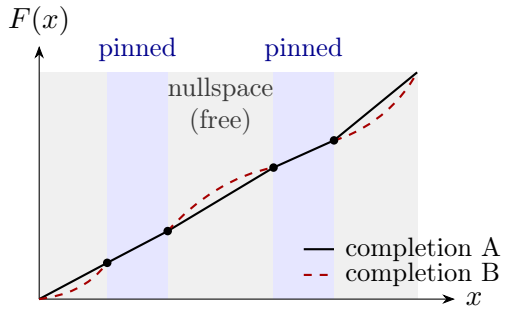

Figure 1: Why feedback repair is underdetermined. A feedback window pins the cumulative mass F at the predicate boundaries it observes (dots); on the pinned intervals (blue) every consistent completion agrees, but on the regions the feedback leaves free (grey)—the nullspace—completions may differ (solid versus dashed) while still matching every observation. OASIS learns which completion to place in the free regions; classical methods fill them with a fixed default.

whose free dimension is $(C-1)-r$. We call this free part the *nullspace* that the feedback leaves.

A deployed histogram exposes only $B-1$ of these C cell boundaries, and on a fixed grid the two representations are interchangeable—inverting the implied CDF recovers the cell masses from the exposed quantiles. Section 4 therefore measures an *operational* rank on those $B-1$ boundaries—a deployable proxy for rank(A)—and turns this nullspace view into the pinned-regime result, measuring when a prior can affect the projected marginal.

This formulation is deliberately single-column. OASIS does not infer multi-column dependence from single-column feedback, and it does not repair join-key dependence, physical design, or cost-model calibration. The single-column marginal is nonetheless a load-bearing input to the estimators built on top of it: copula, IPF, and factorized-join models all factor the joint distribution into one-dimensional marginals and a dependence structure, so a stale marginal biases the joint regardless of how accurately that structure is modeled. Correcting the marginal therefore propagates to any consumer that composes it—as our composition-family (Section 6.2.1) and FactorJoin (Section 6.2.2) experiments show by holding the dependence structure fixed and varying only the marginal. Where cross-column correlation or join-key skew is the dominant *residual* error, an external dependence model remains necessary, and OASIS is complementary to such a model rather than a replacement. We treat the relative size of these error sources as a stated

assumption, not a measured claim about any production system.

3. OASIS Design

3.1. System Overview

OASIS is a middleware that sits at the statistics layer, between the statistics store and the optimizer, and is organized as three layers (Figure 2). The *conversion layer* (L1, Section 3.2) maps the engine's native statistics object to a canonical one-dimensional CDF view and back, so the rest of the system is engine-independent. The *imputation layer* (L2, Section 3.3) produces a marginal prior that completes the feedback nullspace; it is the learned component, and we refer to it interchangeably as Stage 1. The *calibration layer* (L3, Section 3.4) projects the prior onto the active feedback and routes among candidate marginals; we refer to it as Stage 2.

When the optimizer requests a histogram, the request passes down through the three layers and back (Figure 2). Two integration points suffice: a query-completion listener that records predicate-level feedback from executor row counts, and a statistics-assembly hook that substitutes the corrected histogram before it reaches the optimizer. Because the correction is inserted at the statistics layer, the optimizer's search procedure and cost model are untouched, and feedback collection adds only bounded per-conjunct bookkeeping.

OASIS names the approach—the learned prior completed by feedback projection—and in its default deployed form applies the hard projection. Its internal candidates—the unprojected prior (OASIS-noProj) and a consistency-gated Soft projection—and the residual-gated Router that selects among {stale, ISOMER, OASIS-noProj, OASIS, Soft} are detailed in Stage 2 (Section 3.4). The two deployable recommendations are OASIS (default) and Router (when a deployment-visible fallback is wanted). The external baselines are the stale histogram and ISOMER (the maximum-entropy projection initialized from stale), together with the self-tuning histograms STHoles and QuickSel-H.

3.2. Statistics-Format Conversion

Many engines do not expose a standalone editable equi-depth histogram. PostgreSQL, for example, couples histogram boundaries with MCV lists and other per-column summaries. We instantiate the interface for PostgreSQL-style statistics with a three-phase adapter: reconstruct a canonical distribution view from the native object, apply correction on the canonical view, and

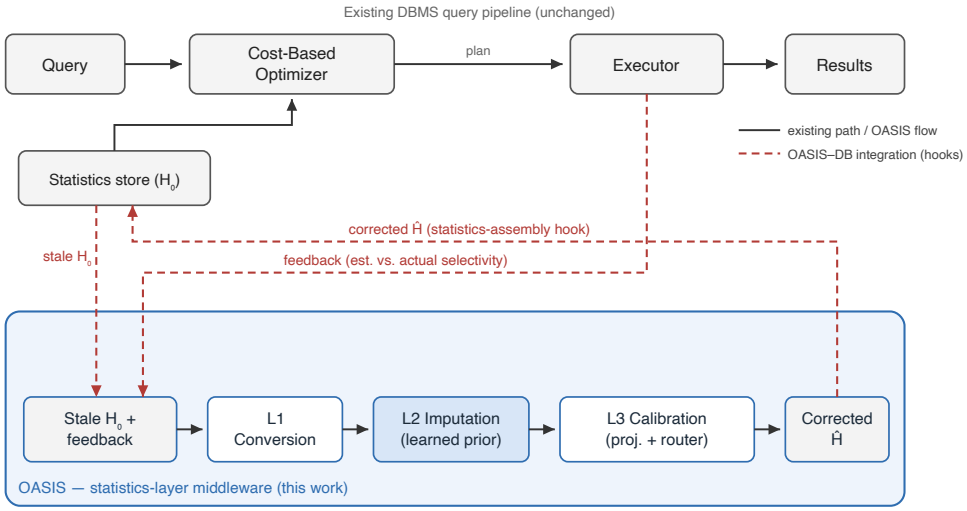

Figure 2: OASIS as a statistics-layer middleware. The existing DBMS query pipeline (solid, top) is left unchanged. OASIS runs its own input-to-correction flow (solid): it reads the stale histogram H_0 and a recent feedback window, then applies conversion (L1), imputation (L2), and calibration (L3) to emit a corrected histogram $\hat{H}$. It attaches to the pipeline only through the dashed integration hooks—tapping predicate feedback from the executor and substituting $\hat{H}$ into the statistics the optimizer reads. Only L2 is learned; L1 and L3 are deterministic.

decompose the corrected view back into native fields. The round trip preserves total mass to machine precision (residual-quantile MAE mean 2.4×10^{-17}, max 9.2×10^{-17}), keeps selectivity estimates consistent (global MAE 1.1×10^{-4}, worst case below 0.39%), and completes in 0.066–0.073 ms.

Only the conversion layer is engine-specific; the imputation and calibration layers operate on the canonical CDF and are unchanged across systems. Porting OASIS to a new engine therefore reduces to writing one adapter. The effort depends on how the engine exposes single-column statistics. MySQL and MariaDB keep equi-height/equi-width histograms in the data dictionary and expose them through `ANALYZE TABLE ... UPDATE HISTOGRAM` and `information_schema`, which maps to the canonical CDF almost directly. Microsoft SQL Server stores step-based histograms readable via `DBCC SHOW_STATISTICS`, again a close fit. Oracle couples height-balanced and hybrid histograms with column-level density and is closest to the PostgreSQL case we implement, where the adapter must also reconcile a most-common-

value list. Engines that do not expose an editable histogram object would require correcting statistics through their maintenance API rather than in place. We validate the PostgreSQL adapter end-to-end; the others are design targets, not claims, but they share the same canonical interface.

3.3. Stage 1: A Prior Trained Against Optimizer Error

The Stage-1 component supplies the nullspace completion. The central design choice is the *training objective*, not the architecture: we train the prior so that, after feedback projection, the resulting marginal is a good optimizer input—not so that it reconstructs the post-drift histogram.

The prior generator consumes a fixed tensor with four blocks: normalized stale quantile boundaries, lightweight metadata, up to K recent predicate-feedback observations, and a validity mask. We use B=10 and K=16, small enough for planning-time use. The generator is a compact permutation-invariant set encoder over the stale-quantile and feedback tokens with a residual head that predicts corrections to interior quantile boundaries; the output is clamped and monotonized to a valid CDF. Permutation invariance over feedback tokens matches the unordered nature of an observation window and lets the same model consume any K.

Let p_θ be the prior marginal and Π the feedback projection. We train θ to minimize

$$\begin{aligned}\mathcal{L} = w_{\text{fut}}\,\mathrm{QErr}_{\log}\big(\Pi(p_\theta), \sigma_{\text{fut}}\big) + w_{\text{join}}\,\mathrm{Regret}_{\mathrm{FJ}}\big(\Pi(p_\theta)\big) \\ + w_{\text{cons}}\,\rho_{\text{fb}}(p_\theta) + w_{\text{reg}}\,\|p_\theta - p_{\text{fresh}}\|_1,\end{aligned} \tag{2}$$

where $\mathrm{QErr}_{\log}$ is the held-out future-predicate log-Q-error, $\mathrm{Regret}_{\mathrm{FJ}}$ a differentiable FactorJoin bin-mass join regret, ρ_{fb} the in-window feedback residual, and the last term a light reconstruction regularizer toward the post-drift marginal. The first two terms make the objective *optimizer-facing*: the model is rewarded for the selectivity and join estimates the projected marginal induces, evaluated on predicates it has not seen, not for matching a histogram. Training uses a sparse-to-dense feedback curriculum and a domain-randomized mixture of drift families so the completion generalizes across feedback budgets and drift mechanisms; Section 6.1.3 shows that the reconstruction and consistency regularizers, not the optimizer terms alone, are what keep the dense-feedback regime from collapsing.

Two caveats: Π appears inside the loss, but the ablation (Section 6.1.3) shows post-hoc projection performs as well, so "calibration in the loop" is not

the mechanism; and the reconstruction term is only a regularizer—trained on reconstruction *alone*, the model is worse than the classical priors. The prior is trained offline and reused across columns without per-table retuning.

3.4. Stage 2: Feedback Projection and the Router

Stage 2 completes the prior in the feedback-constrained subspace and decides, per case, whether to trust the learned completion or fall back to the maximum-entropy one.

The hard operator adapts the ISOMER/IPF feedback-consistency projection [15] to act on a supplied prior: it builds the interval partition induced by the active feedback predicates, initializes cell masses from the prior, and performs cyclic I-projections until every active predicate's interval mass matches its observed selectivity. When sequential drift makes the active set inconsistent, the oldest constraints are dropped first. The operator is not new; its role here is to realize the completion picture of Section 2.3—it pins the marginal exactly on the feedback span and leaves the learned prior to occupy the nullspace—and to provide the safe form for joins and the planner.

On future single-column predicates, matching a handful of recent intervals exactly can overfit; a soft operator that minimizes $\mathrm{KL}(p \,\|\, p_{\text{prior}}) + \lambda \sum_j w_j (A_j p - y_j)^2$ over the same partition generalizes slightly better, with $w_j = \exp(-(\text{conflict}_j/\tau)^2)$ down-weighting feedback that the most recent observations contradict. A banded relaxation of the equality constraints, by contrast, monotonically worsens accuracy as the band widens, so the soft operator's small edge comes from better cell-level conditioning, not from looser matching. We keep Soft as a router candidate but, given the ablation (Section 6.1.3), do not build the contribution on it.

No single operator dominates: hard projection is safest on joins and the planner, while the learned completion wins on sparse single-column feedback. The Router selects, per case, the candidate with the lowest *in-window feedback residual*: the mean absolute discrepancy, computed over the *full* recent observation window, between a candidate's estimate and the observed selectivity.

This signal discriminates among candidates for two reasons, even though hard projection matches its active constraints exactly. First, the pool is not uniformly exact-matching: the stale histogram and the raw prior (OASIS-noProj) apply no projection and leave a large residual, and the Soft candidate matches only approximately, so the residual cleanly separates these from the projected candidates. Second, the projection enforces only the *consistent*

active subset—under sequential drift the stale-feedback filter drops the oldest constraints when the active set becomes inconsistent—so the full-window residual measures how well each candidate fits the constraints just outside its own enforced set, where ISOMER (seeded from stale) and OASIS (seeded from the learned prior) differ. Between candidates that enforce the same active set the residuals are nearly identical, and ties break deterministically toward the safer projection.

The gate never observes test predicates, true cardinalities, runtimes, or plan shapes, so it is non-oracle, and because it minimizes residual over a pool containing the hard projection, it cannot increase the in-window residual relative to that pool. This guarantees a property of the *residual*, not of plan shape: what it buys empirically, on the workloads we evaluate, is a fallback that reverts to the maximum-entropy baseline wherever the learned completion over-extrapolates (Section 6.2), selecting the learned completion where free degrees of freedom exist and reverting to ISOMER once the marginal is pinned; a plan-shape-aware gate is left to future work (Section 8). A diagnostic over the proxy cases (supplementary material) confirms the Router beats always-ISOMER, always-OASIS, and a random projected choice and approaches an offline oracle in aggregate, while its residual signal agrees with the oracle's per-case choice only 42% of the time and does not bound the worst case—the evidence behind treating it as a fallback, not a guarantee.

4. Analysis of the Feedback Nullspace

Section 2.3 framed feedback repair as filling the free degrees of freedom the feedback leaves—its nullspace. That same view bounds what any learned completion can do, and lets us quantify how the free dimension shrinks as feedback accumulates.

In the *pinned regime* this bound becomes exact. Fix the cell-mass representation above and assume the feedback is consistent and noise-free. Consider any method that returns a consistent marginal by applying the *same* hard projection Π (the I-projection onto $\{m \in \Delta : Am = y\}$) to its own prior. If the constraints identify the marginal within this representation (full rank, $r = \dim \Delta$), then the consistent set is a single point, and *every such method returns that same marginal regardless of its prior*; in particular ISOMER and the projection of the OASIS prior coincide. Two priors can differ in their projected output only on the nullspace of A, of dimension $\dim \Delta - r$.

Table 1: Feedback-constraint rank and residual free degrees of freedom (DOF) of the marginal versus feedback budget K. We report an *operational* rank on the $B-1 = 9$ exposed histogram boundaries—a deployable proxy for the exact cell-level rank(A) (Section 2.3). Free DOF is the nullspace dimension the projection leaves for the prior to complete; once feedback saturates the DOF the marginal is pinned and every method applying the same projection coincides (the pinned regime).

K	mean rank	mean free DOF	marginals pinned
2	2.32	6.68	0%
4	4.65	4.35	0%
6	6.95	2.05	6%
8	8.71	0.29	73%
12	8.98	0.02	98%
16	8.98	0.02	98%

This pinned-regime result is simple but central: the entire opportunity for a learned prior lives in the nullspace, and the opportunity vanishes continuously as feedback saturates the constraints. ISOMER fills the nullspace with maximum entropy [15]; STHoles and QuickSel-H fill it with their own structural assumptions. It concerns *any prior closed by the same projection*, not the native STHoles or QuickSel-H algorithms, which need not reduce to this I-projection. Our evaluation makes the comparison like-for-like by applying the same projection operator to every prior (Section 6.1.1), the setting in which the pinned-regime coincidence holds. The question OASIS answers is what completion minimizes *downstream optimizer error*.

Before the empirical ladder, we make this prediction quantitative, because it governs how every later result should be read. An equi-depth marginal with B=10 buckets exposes $B - 1 = 9$ free interior boundaries. For each held-out case we count the independent feedback constraints (distinct interior boundary anchors, merged within tolerance) and the residual free degrees of freedom (DOF); as Section 2.3 notes, this is an operational rank on the exposed boundaries, a proxy for the exact cell-level rank(A) rather than rank(A) itself. Table 1 reports the mean rank and free DOF as a function of the feedback budget K.

The free DOF collapses from 6.68 at K=2 to ≈ 0 once the feedback constraints saturate the $B - 1$ boundaries—here by $K \geq 12$ on this constraint-spanning predicate population, with 73% of marginals already pinned at K=8. The nullspace is therefore a property of *which regions of the domain*

the feedback constrains, not merely of how many observations there are: a finite window pins the marginal near the observed predicates and leaves it free elsewhere. This predicts two signatures the rest of the paper confirms. (i) *Spatially*, the learned completion should win most on predicates that fall far from any feedback, directly in the nullspace, and least where the feedback already pins the marginal; the feedback-locality bins (Figure 3) measure exactly this. (ii) *At saturation*, when a feedback window densely covers the queried region and drives the rank to $B - 1$, the marginal is pinned and OASIS must converge to ISOMER in the pinned regime; the dense PostgreSQL planner regime (Section 6.2.3) is where this tie appears. A prior given more feedback context fills the *residual* nullspace better, not less, so the completion's advantage persists—and even grows—across feedback budgets (Figure 4) until the constraints actually saturate.

5. Experimental Methodology

The evaluation is primarily *planner-facing*: we measure the corrected statistics object together with the row estimates and plan shapes a real PostgreSQL optimizer derives from it, plus one deliberately narrow TPC-H runtime check confirming that calibrated statistics track fresh-statistics execution time. We do not claim broad query-runtime superiority over the stale baseline.

The evaluation follows the deployment path of a marginal statistic. We first establish the corrected single-column object: the prior carries optimizer-relevant drift signal, its advantage is concentrated in the nullspace as Section 4 predicts, the gain comes from the composite objective, and the same calibration works for non-OASIS priors. We then probe external validity, widening from synthetic drift to unseen families, event streams, and a real trace. Finally we ask whether the corrected marginal stays safe once downstream estimators and a real planner consume it.

Unless stated otherwise, the prior is trained on a domain-randomized drift mixture and evaluated on held-out drift intensities $q \in \{1, 3, 5, 10, 15, 20, 25, 30\}$; per-intensity tables report a representative subset of this grid (e.g., $q \in \{1, 5, 10, 20, 30\}$). All feedback-driven methods consume the same observation budget K=16 and emit corrected quantiles on the same B=10 grid. We group the baselines into three classes. (i) *True self-tuning histograms*: STHoles [13] and ISOMER [15], run as their published feedback-driven algorithms. (ii) An *adapted histogram-interface baseline*:

QuickSel-H [16], which exposes QuickSel's density estimate on the shared B=10 grid; it is a faithful adaptation to the common output interface, not a line-by-line reproduction (the adaptation is detailed in the supplement). (iii) An *internal learned query-driven control*, LQM, which fits a monotone sigmoid-basis network to the same feedback window online, in the style of learned query-driven estimators [17, 18] but restricted to the single-column statistics-repair setting and the same no-rescan inputs. Because QuickSel-H and LQM are matched-interface stand-ins rather than the published systems, we draw no claim of superiority over learned cardinality estimators in general; they calibrate where a flexible feedback-only fit lands relative to the trained completion. In result tables, the *OASIS* column is its default deployed form and OASIS-noProj the unprojected control; some tables label the residual-gated selector *Hybrid* and others *Router*—the same selector, to be read as the deployed router.

Every method starts from the *same* stale statistics, sees the *same* feedback window, is projected onto the *same* constraints, and is evaluated on the *same* held-out split; no method receives oracle access, and "fresh" is computed from the post-drift data only as an accuracy upper bound. Table 2 defines the metrics used throughout. For sensitivity diagnostics we also use a transparent optimizer-decision proxy: future predicates are drawn from held-out distributions, each statistics state selects scan and join choices under a fixed cost proxy, and the choice is scored by its true proxy cost. The proxy analyzes feedback-budget and noise behavior; it is not a runtime benchmark.

Drift model and its justification. We do not have access to a production stream of statistics-refresh histories, so we drive staleness with controlled drift, chosen to mimic how analytical tables actually evolve between `ANALYZE` passes. The drift intensity q scales how far the post-drift distribution moves from the snapshot H_0 (in units of inter-quantile shift), so larger q corresponds to a longer or more turbulent refresh interval; we sweep $q \in \{1, \ldots, 30\}$ to cover mild to severe staleness. The drift *families* (batch loads, range and date-band shifts, skew evolution, outlier bursts, multimodal and seasonal mixtures) are the update patterns that workload-evolution and learned-estimator drift studies report as the practical causes of estimator degradation [19, 20, 11, 21], and the DML traces of Section 6.1.4 reproduce several of them as explicit insert/update/delete streams. We treat the drift model as a stated, transparent assumption rather than a claim about any single production system.

Table 2: Evaluation metrics. Direction marks whether lower (↓), higher (↑), or near-one ($\to 1$) is better. $\hat{s}, s^*$ are estimated and actual selectivity; p is the corrected marginal; $A_j p$ is its mass on feedback interval j with observed value y_j.

Metric	Definition	Dir.
Q-error	$\max(\hat{s}/s^*, s^*/\hat{s})$, Eq. (1)	↓
Selectivity MAE	mean $\lvert\hat{s} - s^*\rvert$ over predicates	↓
Quantile MAE	mean $\lvert v_i - v_i^{\text{fresh}}\rvert$ over boundaries	↓
Feedback residual	mean $\lvert A_j p - y_j\rvert$ on the observation window	↓
Row Q-error	Q-error of `EXPLAIN` estimated rows vs `COUNT(*)`	↓
Fresh-plan match	frac. of queries whose plan equals the fresh-stats plan	↑
Recovery	frac. of stale/fresh plan disagreements the method fixes	↑
New deviations	frac. of stale/fresh agreements the method breaks	↓
Join regret	selected-plan proxy cost / optimal proxy cost	↓
Time/Fresh	geomean warm-cache runtime ratio to fresh stats	$\to 1$

6. Empirical Evaluation

6.1. Single-Column Correction

6.1.1. The Learned Completion versus Classical Priors

We hold the projection operator and feedback constraints fixed and vary only the marginal that initializes the projection: stale (which yields ISOMER), the self-tuning histograms STHoles and QuickSel-H, the learned OASIS prior, and the post-drift fresh marginal as an upper bound. All variants are scored on held-out future predicates.

Tables 3 and 4 give the central single-column result. Initializing the same projection from the learned completion lowers aggregate future-predicate Q-error from 1.523 (ISOMER) to 1.328, a 12.8% reduction, and—unlike a reconstruction-trained prior—it also beats the classical self-tuning histograms, by 6.1% over STHoles (1.415) and 5.7% over QuickSel-H (1.407). The advantage grows with drift, reaching +15.2%, +22.2%, and +19.6% over ISOMER at q=10, 20, and 30, and the fresh-initialized lower bound (1.151) confirms headroom remains. That the learned completion beats two strong classical completions under the *same* projection shows that the gain is in what fills the nullspace, not in how the feedback is matched.

6.1.2. The Gain Lives in the Nullspace: Distance and Budget

Section 4 predicts the learned completion should help most where feedback did not reach. Two complementary cuts confirm this—across *space* (a

Table 3: Projection-initialization ablation on held-out future predicates (in-distribution compound drift; geometric-mean selectivity Q-error). The projection operator and feedback constraints are identical across columns; only the marginal that initializes the projection changes. **ISOMER** initializes from the stale histogram and is exactly ISOMER; **OASIS** initializes from the learned stage and is the OASIS; **Fresh-init** initializes from the post-drift marginal as a reference upper bound. Win frac is the fraction of held-out predicates on which the OASIS beats ISOMER.

q	Stale	ISOMER	OASIS-noProj	OASIS	Fresh-init	Fresh	OASIS vs ISOMER	Win frac
1	1.209	1.142	1.174	1.148	1.104	1.000	-0.6%	50.7%
5	1.968	1.394	1.419	1.318	1.163	1.000	+5.4%	61.5%
10	2.990	1.617	1.464	1.372	1.176	1.000	+15.2%	70.1%
20	3.253	1.788	1.572	1.391	1.156	1.000	+22.2%	69.5%
30	3.193	1.779	1.624	1.429	1.159	1.000	+19.6%	68.9%
All	2.364	1.523	1.442	1.328	1.151	1.000	+12.8%	64.1%

Table 4: Projection-initialization with classical priors. The hard projection and feedback constraints are identical across columns; only the marginal that initializes the projection changes. **ISOMER** initializes from the stale histogram; **STHoles-init** and **QuickSel-init** initialize from those self-tuning-histogram priors; **OASIS** initializes from the learned MLP; **Fresh-init** is the post-drift upper bound. Last two columns are the OASIS-init improvement over the classical inits (positive favors OASIS).

q	Stale	ISOMER	STHoles-init	QuickSel-init	OASIS	Fresh-init	OASIS vs STHoles	OASIS vs QuickSel
1	1.209	1.142	1.147	1.165	1.148	1.104	-0.1%	+1.4%
5	1.968	1.394	1.344	1.358	1.318	1.163	+1.9%	+2.9%
10	2.990	1.617	1.554	1.528	1.372	1.176	+11.8%	+10.2%
20	3.253	1.788	1.565	1.484	1.391	1.156	+11.2%	+6.3%
30	3.193	1.779	1.510	1.540	1.429	1.159	+5.3%	+7.2%
All	2.364	1.523	1.415	1.407	1.328	1.151	+6.1%	+5.7%

predicate's distance to feedback) and across *budget* (the size of the feedback window). The first cut bins each held-out predicate by the directed Hausdorff distance from its endpoints to the nearest feedback endpoint in fresh-CDF space.

Figure 3 confirms the prediction: OASIS's gain over ISOMER rises monotonically from 9.1% near feedback (distance ≤ 0.05), through 14.5% in the mid band, to 21.2% far inside the nullspace (> 0.15)—the spatial signature of a completion, where the constraints fix the marginal near observations and the learned prior supplies the rest.

The second cut varies the feedback budget K directly, in the optimizer-decision proxy, where future predicates are spread across the domain so that

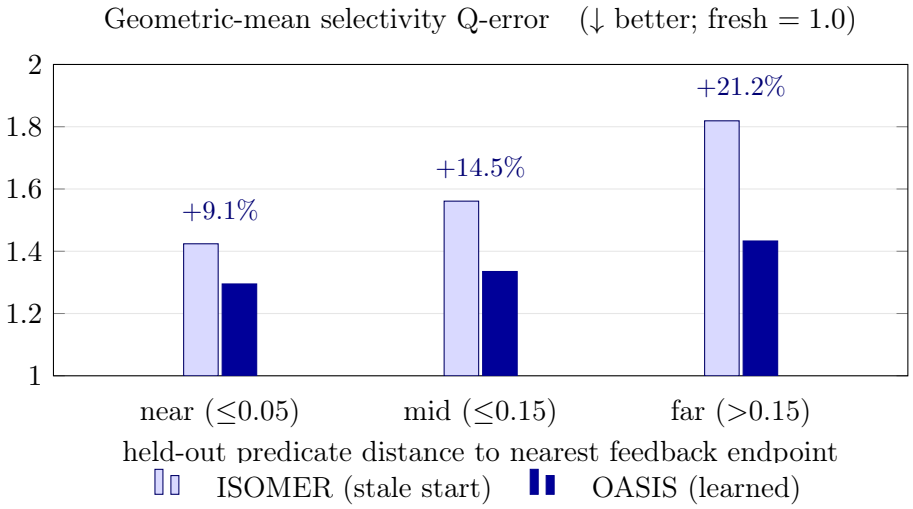

Figure 3: Feedback-locality diagnostic: geometric-mean selectivity Q-error for ISOMER and the learned OASIS completion, with held-out predicates binned by distance from the nearest feedback endpoint (fresh = 1.0 is the ideal floor). Both degrade as predicates move into the nullspace, but OASIS stays lower and its advantage over ISOMER grows monotonically—+9.1% near feedback, where the projection already pins the marginal, to +21.2% far inside the nullspace—the spatial signature of a completion.

a finite window leaves residual nullspace at every budget.

Figure 4 shows the learned completion beats ISOMER at *every* budget: +4.3% at K=2 (1.994 versus 2.083), +6.8% at K=4, +12.1% at K=8, and +15.7% at K=16 (1.402 versus 1.664). On this proxy surface ISOMER degrades gracefully as feedback thins, so the relative gap stays modest at every budget. On the real PostgreSQL planner the maximum-entropy completion instead fails sharply at sparse feedback, and there the gap is largest—a 2.7× aggregate row-Q-error difference at K=2 over twelve configurations, up to 5.2× on a representative one (Table 10). The two pictures agree on the mechanism: the learned completion earns its advantage wherever feedback under-determines the marginal, and the methods converge only when dense feedback saturates the constraints and pins it. This is also why the gain concentrates spatially in the nullspace (Figure 3) rather than at any particular budget.

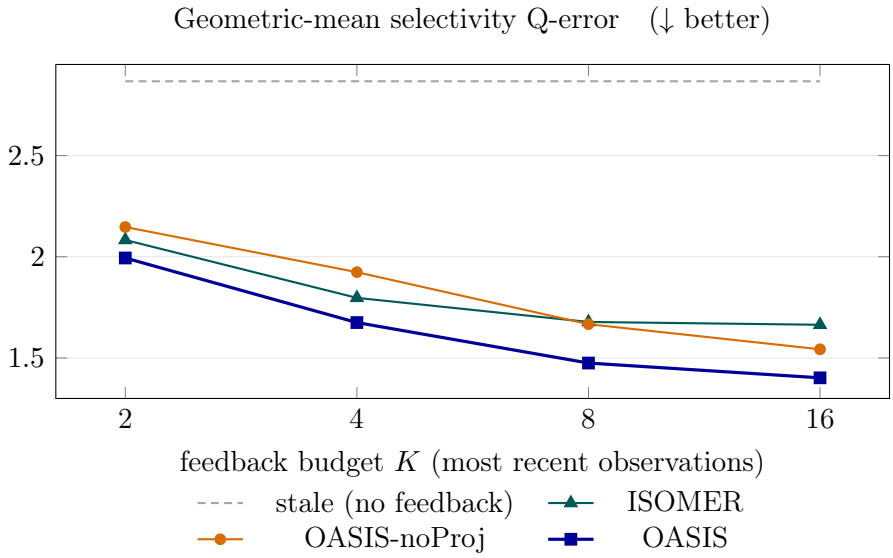

Figure 4: Feedback-budget sweep in the optimizer-decision proxy: geometric-mean selectivity Q-error as the feedback window grows (each method sees only the K most recent observations; stale uses none). The learned completion stays lowest at every budget, and its advantage over ISOMER persists—and even grows—as feedback densifies (+4.3% at K=2 to +15.7% at K=16): more feedback fills the residual nullspace rather than closing the gap, the rank curve the analysis predicts.

6.1.3. The Gain Is the Objective, and It Is Estimator-Agnostic

We now decompose the objective of Eq. (2), holding data, capacity, and projection fixed. Each variant's prior is calibrated by the *same* real projection on held-out future predicates and reported as percentage improvement over the strongest classical prior (STHoles) at three feedback budgets.

Three conclusions follow. First, reconstruction-only training fails at every budget: training a prior to look like the post-drift histogram makes it worse than the classical priors, so an optimizer-facing objective is necessary. Second, the composite objective creates the gain, and the reconstruction and consistency regularizers are essential at dense feedback: future-loss alone, and future-loss+join, collapse to a tie at K=16, and only the regularizers restore the dense-feedback margin. Third, *training the prior through the projection is not the source of the gain*: "raw-train + post-hoc projection," where the loss never sees the projection, matches "full in-loop" to within roughly one percentage point at every budget (Table 5), and the equivalence holds across two training seeds on the full drift range. We therefore do not claim

Table 5: Stage-1 objective ablation (held-out future-predicate Q-error after the same real projection; % improvement over STHoles, higher is better). All variants share data, capacity, and projection. *Reconstruction-only* is the floor; the composite objective drives the gain; *raw-train + post-hoc projection* matches *full in-loop*, so differentiable projection *in the loop* is not the source of the gain. The dense-feedback (K=16) column shows the reconstruction/consistency regularizers, not in-loop projection, are what prevent collapse.

Stage-1 objective variant	K=2	K=6	K=16
Histogram reconstruction only	−1.2	−1.9	−7.8
Future-loss only (in-loop proj.)	+8.8	+7.0	+0.5
+ FactorJoin regret	+8.0	+5.9	−0.0
+ feedback residual	+9.8	+7.4	+3.4
Raw-train + *post-hoc* projection	+9.4	+9.0	+7.8
Full (in-loop projection)	+8.7	+7.9	+7.8

Table 6: Stage-1 prior-source swap under a fixed Stage-2 calibration protocol. Each row supplies a different single-column marginal prior to Stage 2; lower Q-error and feedback residual are better.

Stage-1 source	Raw QE	+Hard QE	+Router QE	Router gain	Raw resid.	Router resid.
Stale prior	2.404	1.539	1.437	40.2%	0.1206	0.0383
LinInterp	2.116	1.508	1.417	33.0%	0.1072	0.0379
FeedAvg	2.474	1.544	1.437	41.9%	0.1256	0.0383
STHoles	1.572	1.436	1.394	11.3%	0.0542	0.0365
QuickSel-H	1.614	1.423	1.392	13.7%	0.0573	0.0347
LQM (learned QD)	1.607	1.454	1.425	11.3%	0.0471	0.0336
OASIS MLP	1.489	1.346	1.328	10.8%	0.0628	0.0354

in-loop calibration is necessary; it is a convenient but inessential training device, and post-hoc projection—simpler to deploy—performs identically. The contribution is the composite objective.

The converse question is whether the completion-plus-projection picture is specific to the OASIS prior. We run a prior-source swap: the same projection and router are applied to stale statistics, two feedback heuristics (LinInterp, FeedAvg), STHoles, QuickSel-H, and the OASIS prior, on the same cached drift cases and feedback windows.

Table 6 shows the projection and router improve every prior, lowering future-predicate Q-error to a tight band (1.33–1.44) and the feedback residual to 0.035–0.038, and the Router is no worse than hard projection for *every* source, including the learned query-driven control LQM (an internal matched-

Table 7: Out-of-distribution drift realism suite. OASIS is trained only on compound drift and evaluated on deployment-inspired drift families. Values are geometric mean selectivity Q-error over held-out cases; Hybrid is the residual-gated selector and **bold** marks the best non-fresh method per family.

Drift family	Stale	ISOMER	OASIS-noProj	OASIS	Soft	Hybrid	Fresh
Batch load	4.203	1.461	1.987	**1.251**	1.422	1.390	1.000
Range shift	1.994	**1.210**	1.381	1.212	1.286	1.221	1.000
Skew evolution	1.540	1.182	1.521	1.171	**1.163**	1.174	1.000
Outlier burst	1.258	1.076	1.139	**1.064**	1.072	1.071	1.000
Multimodal	1.238	1.066	1.136	**1.063**	1.065	1.065	1.000
Seasonal/mixed	1.212	1.066	1.127	**1.058**	1.064	1.063	1.000

interface baseline, not a published learned-CE system; Section 5; raw 1.607, routed 1.425). The OASIS prior is the strongest *raw* input (1.489, below STHoles 1.572, QuickSel-H 1.614, and LQM 1.607), and it routes to the best final value (1.328, versus 1.425 for LQM and 1.39–1.44 for the others). The learned query-driven baseline therefore lands with the classical self-tuning histograms rather than with OASIS: fitting a flexible model to the feedback alone is not enough; the advantage comes from training the completion against optimizer error. This restates the paper's central claim at the level of one experiment: the projection and router empirically make any imperfect prior safe to use on these workloads, while the trained completion is what wins the nullspace where the feedback leaves it open.

6.1.4. Robustness Beyond the Training Generator

All results so far stay inside the training generator. Three steps leave it: unseen drift families, explicit DML event streams, and a real public trace. The first evaluates the prior, trained on the domain-randomized mixture, unchanged on six deployment-inspired drift families: batch loads, range shifts, skew evolution, outlier bursts, multimodal drift, and seasonal/mixed drift.

Table 7 shows the expected graceful degradation. The deployed forms—projected and routed—remain among the strongest non-fresh methods, edging ISOMER on batch-load (1.251 vs 1.461), skew (1.171 vs 1.182), and outlier drift; and where the *raw* prior over-extrapolates on an unseen family—batch-load (1.987) and skew (1.521)—the projection and residual-gated Hybrid pull it back to ISOMER level using feedback residuals alone. Under distribution shift the learned completion no longer dominates, but it never escapes the safety net.

Figure 5: Benchmark-inspired DML traces: percentage reduction in geometric-mean selectivity Q-error relative to stale (cell value; positive is better, negative is a regression below stale). The projected OASIS and the Hybrid track ISOMER and the fresh reference, while the raw OASIS-noProj regresses on update-heavy traces (promotion price, seasonal mixed)—the failure the projection and router are there to absorb.

The second step replaces drift operators with explicit DML event streams anchored to analytical maintenance patterns—append-heavy sales, promotion price revisions, inventory restocking, returns/cancellations, segment churn, seasonal maintenance—each emitting insert/update/delete counts before a feedback observation.

The third uses a real public trace with a deliberately bounded claim: lacking access to private DBMS production logs, we use the NASA Kennedy Space Center HTTP request trace [22, 23], treating each request as an append to an event table and deriving three single-column signals (log reply bytes, time of day, log path length).

Table 8, with Figure 5, reinforces the safety story. On the DML traces the Hybrid tracks ISOMER and stays the best non-fresh choice. On the real NASA trace the boundary is sharper: the raw learned completion regresses in aggregate (1.466 versus stale 1.241), and catastrophically on time-of-day predicates (2.66, worse than stale's 1.77), because a real append stream is not the training drift. The projection and residual-gated Router prevent the failure, recovering to ISOMER level (1.25 on time-of-day) and to the best deployable aggregates (Hybrid 1.114, Router 1.115). This is the deployment layer behaving as intended: trust the learned completion when the feedback window supports it, and fall back when the real trace exposes over-extrapolation.

Table 8: Public production-trace case study using the NASA Kennedy Space Center HTTP request trace. Each request appends one event to an analytics table; stale statistics are built from an early prefix, feedback is collected after later real requests, and future predicate probes include constants drawn from held-out later requests. Each single-column signal (log reply size, request timestamp, request path length) grows $\approx 800\%$ from the stale prefix to the evaluation point. Values are geometric mean selectivity Q-error and **bold** marks the best non-fresh method per signal. This is a public telemetry trace, not a private DBMS query log.

Trace column	Stale	ISOMER	OASIS-noProj	OASIS	Soft	Hybrid	Router	Fresh
Reply bytes	1.056	1.059	1.071	1.063	1.055	1.057	**1.053**	1.000
Time of day	1.767	1.254	2.661	1.342	1.318	**1.251**	1.255	1.000
Path length	1.025	1.057	1.104	1.082	1.065	**1.044**	1.050	1.000
Aggregate	1.241	1.120	1.466	1.156	1.140	**1.114**	1.115	1.000

6.2. Downstream Optimizer Consumption

Section 6.1 corrected the single-column object in isolation; we now feed it to its downstream consumers—composition estimators, a join kernel, and a real PostgreSQL planner—and ask where the correction still holds. The boundary from Section 1 holds: the primary evidence is estimate quality and plan *shape*, with one narrow TPC-H runtime check.

6.2.1. Composition-Family Generalization

We embed OASIS as a drop-in marginal source in six independently designed composition estimators—the independence/maximum-entropy estimator, IPF/Sinkhorn on a 2D grid, and Gaussian, Clayton, Gumbel, and Frank copulas—varying only the marginal input. The dependence structure is held fixed and identical for every marginal, so this is a clean marginal ablation, and the Archimedean copulas are intentionally mis-specified for the Gaussian-correlated data.

Figure 6 shows the same pattern across the family. OASIS-noProj yields weak or negative gains, whereas both projected methods recover most of the error— OASIS by 19.7%–36.0% over stale, comparably to ISOMER. The benefit does not depend on one dependence model and survives the mis-specified copulas: feedback consistency, not the raw prior, is the prerequisite for safe composition.

6.2.2. FactorJoin Integration

To test propagation into a learned *join* estimator, we embed OASIS in the bin-based join kernel of FactorJoin [24]. For an equi-join $A.k = B.k$ it

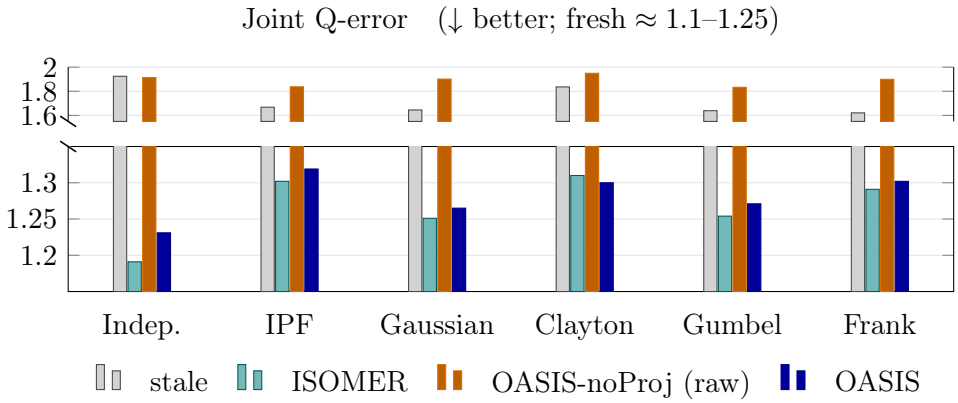

Figure 6: OASIS as a drop-in marginal across six composition estimators (two-column range predicates; dependence structure fixed per estimator, only the marginal varies); joint Q-error on a broken axis (upper panel for the large values, lower panel zoomed on the recovered cluster). Both *projected* methods—the classical ISOMER and the learned OASIS—recover most of the stale joint Q-error and are comparable, whereas the same learned prior *without* projection (OASIS-noProj) stays near the stale level. Feedback consistency, not the raw prior, is the prerequisite for safe composition.

estimates $\sum_{\text{bin}} \text{cnt}_A[\text{bin}]\,\text{cnt}_B[\text{bin}]/\text{dv}[\text{bin}]$, where each per-bin key frequency comes from a single-column key histogram, which we make the OASIS-correctable object. The keys are generated independently, so only the single-column key histogram varies across methods.

Figure 7 shows the strongest calibration failure mode. The raw completion is *actively harmful*, raising aggregate join Q-error to 1.621 versus stale 1.213, because the kernel is bilinear in per-bin mass and amplifies mass-concentration errors. Feedback projection fixes it: OASIS improves over stale at every drift level (+12.0%–+23.2%, +15.3% aggregate), the Hybrid by +16.1%, and both closely track the fresh marginal. The more nonlinear the consumer, the more essential the completion operator becomes.

6.2.3. PostgreSQL Planner-Only Statistics Injection

We validate that corrected statistics affect a real optimizer pipeline. The experiment builds alternative single-column statistics for `fact.x` and injects them into `pg_statistic`; true rows come from `COUNT(*)`, estimated rows and plan shapes from `EXPLAIN (FORMAT JSON)`. No query runtime is measured. The batch covers two drift directions, two table sizes, and three seeds (12 configurations).

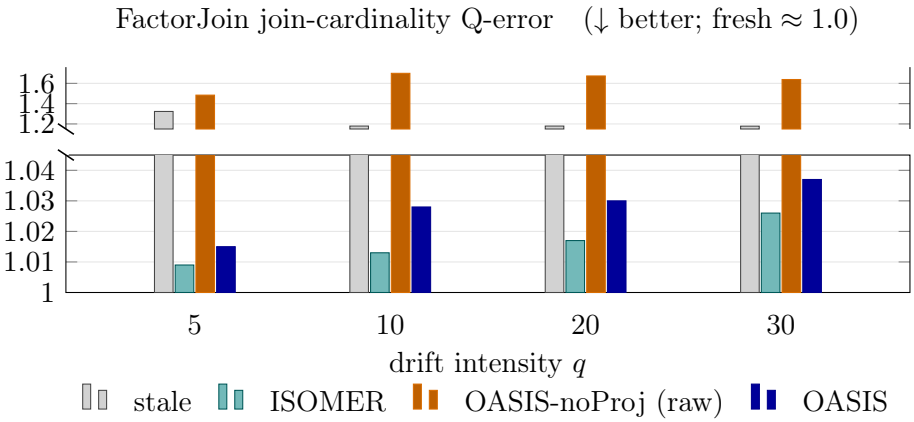

Figure 7: OASIS embedded in the FactorJoin bin-based join kernel as the join-key distribution drifts (only the single-column key histogram varies across methods); join-cardinality Q-error on a broken axis (upper panel for the large values, lower panel zoomed on the near-fresh cluster). The raw prior (OASIS-noProj) is *actively harmful*—its Q-error exceeds even the stale baseline, because the bilinear kernel amplifies mass-concentration errors—whereas both projected methods, the classical ISOMER and the learned OASIS, stay near the fresh marginal (≈ 1.0) at every drift level. The more nonlinear the consumer, the more essential the completion operator.

Table 9 is the strongest DBMS-facing evidence, and its claim is deliberately one of *plan-shape safety*, not accuracy superiority over ISOMER. Pooling over all query instances, stale statistics give aggregate row Q-error 27.768 and match the fresh plan on 56% of queries. (All numbers in this paragraph are pooled over query instances; Table 9 instead reports the per-configuration mean±sd, which differs slightly—e.g. stale row Q-error is 28.453 averaged over configurations versus 27.768 pooled.) OASIS-noProj reduces row Q-error to 3.703 but introduces new plan deviations in 2.3% of cases. OASIS reduces row Q-error to 2.362, matches the fresh plan on 95.9% of queries, recovers 92.1% of stale/fresh disagreements, and cuts new deviations to 1.1%. Crucially, OASIS (2.362) and ISOMER (2.374) are near-identical here—and this is the rank mechanism, not a disappointment: these planner-facing windows are dense enough to pin the active constraints (Section 4), so hard projection converges to nearly the same marginal whichever prior seeds it, and the Router selects ISOMER on all twelve configurations. The planner is the regime where the *completion operator*, not the prior, does the work; the learned prior's advantage instead appears where the feedback leaves a nullspace, which on this same planner is the *sparse*-feedback regime we turn to next.

Table 9: Multi-configuration PostgreSQL planner-only statistics-injection evidence. Values are means over configurations; parentheses show one standard deviation across configurations. True rows use `COUNT(*)`; estimated rows and plan shapes use `EXPLAIN (FORMAT JSON)`. No query runtime is measured.

Method	Row QE	Fresh Plan	Recovery	New Deviations
Stale	28.453 (6.576)	56.1% (5.1)	0.0% (0.0)	0.0% (0.0)
ISOMER	2.390 (0.286)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
OASIS-noProj	3.894 (1.252)	90.9% (5.1)	80.7% (14.4)	2.4% (1.9)
OASIS	2.377 (0.269)	95.9% (1.9)	91.7% (3.8)	1.0% (1.0)
Soft	2.452 (0.312)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
Router	2.386 (0.288)	96.1% (2.0)	92.1% (4.1)	1.0% (1.0)
Fresh	1.415 (0.092)	100.0% (0.0)	100.0% (0.0)	0.0% (0.0)

Sparse feedback on the real planner. The 12-configuration batch above uses the standard dense window (K=16). To expose the nullspace effect, we re-run the *same* 12-configuration grid (left/right drift × 100K/200K rows × 3 seeds, 84 queries each) while varying the feedback budget K (Table 10). At K=2 the maximum-entropy projection degrades sharply—aggregate row Q-error 7.53, a fresh-plan match of only 70.0%, and new plan deviations on 3.7% of queries—while the learned completion holds at 2.83 (Router 2.82), a 92.7% fresh-plan match, and 1.1% new deviations: a 2.7× aggregate row-Q-error gap on a real optimizer, not a proxy. As feedback densifies the constraints pin the marginal and the gap closes: by K=8 the methods are within 2% (ISOMER 2.45, OASIS 2.40), and at K=16 they tie (2.36 vs 2.36), reproducing the dense batch above. The effect is sharper still on individual configurations: on a representative left-shift, 40K-row config the gap reaches 5.2× at K=2 (ISOMER 10.12 versus OASIS 1.96, Router 1.95, with *zero* new deviations). This is the deployment knife-edge the paper's central claim predicts: on the planner, the learned imputation matters most when feedback is too sparse to pin the statistics, and it does not materially regress when feedback is dense.

6.2.4. TPC-H DML-Drift Runtime Sanity Check

Finally, a deliberately narrow runtime check, with its claim stated up front: calibrated statistics *reproduce fresh-statistics behavior*—accuracy, plan shape, and runtime—rather than beat the stale baseline on wall-clock. We load TPC-H SF10, drift it with an RF1-style insert stream that shifts `lineitem.l_shipdate` and `orders.o_orderdate` into a new date band, and

Table 10: Multi-configuration PostgreSQL planner-only sparse-feedback sweep (12 configurations: left/right drift × 100K/200K rows × 3 seeds, 84 queries each), aggregated over all query instances per feedback budget K. Same `pg_statistic` injection path as Table 9; K=16 reproduces that dense batch. Row Q-error (lower better) with new-plan-deviation rate in parentheses (lower better). At sparse K the maximum-entropy projection (ISOMER) degrades while the learned completion (OASIS) and the Router hold; the methods converge as feedback densifies.

K	Stale	ISOMER	OASIS	Router	Fresh
2	27.7	7.53 (3.7%)	2.83 (1.1%)	**2.82 (1.1%)**	1.41
4	27.8	2.95 (1.1%)	**2.49 (2.8%)**	2.50 (3.2%)	1.42
6	27.8	3.44 (1.1%)	2.46 (1.1%)	**2.46 (1.1%)**	1.41
8	27.8	2.45 (1.1%)	**2.40 (1.1%)**	2.41 (1.1%)	1.41
16	27.7	2.36 (1.1%)	2.36 (1.1%)	**2.35 (1.1%)**	1.41

inject single-column statistics for those columns exactly as above, reusing the *same* pretrained prior with no TPC-H retraining (also a cross-schema generalization test). For six date-sensitive queries we record scan row Q-error, plan shape, and warm-cache `EXPLAIN ANALYZE` time over three refresh-stream seeds. Because warm-cache runtime is determined by the induced plan, we time each distinct (query, plan) once at full repetition and assign it to every method sharing that plan.

Table 11 reports the result. Accuracy is unambiguous: stale scan Q-error 5.23 ± 0.24 is repaired to 2.02 for OASIS and 2.01 for the Router, matching fresh (2.01), on a standard schema with no retraining. Calibrated statistics track the *fresh*-statistics runtime to within a few percent (time-to-fresh 0.96 for OASIS, 1.01 for the Router) because they induce the fresh plans. The *stale* wall-clock baseline is not a reliable target: here the stale mis-estimate happens to pick a cheaper-to-run plan, so calibrated and fresh statistics run $\approx 1.5\times$ slower than stale—the price of the correct plan, paid equally by fresh—whereas in a larger-drift configuration the same mis-estimate produced a plan $1.56\times$ slower than fresh. The deployment-relevant invariant is that calibrated statistics track fresh statistics in both accuracy and runtime, while stale statistics make the wall-clock outcome unpredictable in either direction. This is a single-schema check on six queries; we draw no broader runtime-superiority conclusion.

Because a median or geomean can hide a bad tail, we report the runtime distribution against the deployment target, fresh statistics, over all 18 query

Table 11: TPC-H SF10 DML-drift study, aggregated over three independent refresh-stream seeds (mean $\pm$ std; six curated date-sensitive queries each; planner-only statistics injection, PostgreSQL 14). The pretrained OASIS prior is reused with no TPC-H retraining. *Scan Q-err* is the row-estimation error on the drifting date column. *Time/Fresh* and *Time/Stale* are geometric-mean ratios of warm-cache `EXPLAIN ANALYZE` execution time to the fresh- and stale-statistics runs. Calibrated statistics (OASIS, Router) track fresh statistics within the reported distribution (median Time/Fresh ≈ 1, worst case ≤ 1.40) and induce no plan-shape deviation from fresh across the 18 query instances. The stale wall-clock baseline is not a reliable target: a stale mis-estimate can pick a coincidentally cheaper-to-run plan (Time/Stale > 1 here), whereas in other configurations it picks a much slower one; we therefore make no runtime-superiority claim and report that OASIS reproduces fresh-statistics behavior.

Method	Scan Q-err	Time/Fresh	Time/Stale
Stale	5.23 ± 0.24	0.65 ± 0.07	1.00 ± 0.00
ISOMER	2.02 ± 0.08	0.98 ± 0.07	1.51 ± 0.07
OASIS-noProj	2.66 ± 0.07	1.02 ± 0.02	1.59 ± 0.18
OASIS	2.02 ± 0.10	0.96 ± 0.04	1.49 ± 0.11
Router	2.01 ± 0.10	1.01 ± 0.02	1.57 ± 0.15
Fresh	2.01 ± 0.07	1.00 ± 0.00	1.56 ± 0.17

instances (six queries, three seeds). Calibrated statistics induce the fresh plan on every instance—*zero* plan deviations from fresh for OASIS, the Router, and ISOMER—so there is no plan-shape tail. The runtime-to-fresh ratio stays tight: median 0.96 and worst-case 1.23 for OASIS, median 1.00 and worst-case 1.40 for the Router. The only large ratio appears against *stale*, on the single query (TPC-H Q14) whose plan changes stale→fresh: under stale's coincidentally cheap plan it runs much faster—up to $\approx 13\times$ in the worst instance—but fresh statistics show a comparable gap there, so this is the cost of switching to the correct plan, not a regression introduced by calibration. No calibrated method is slower than fresh by more than $1.4\times$ on any query or seed.

6.2.5. The Optimizer-Decision Proxy and Feedback Robustness

The proxy of Section 5 summarizes the regime structure on plan choices. Table 12 reports selectivity Q-error and join regret for each statistics state.

The ordering matches the rest of the paper: stale (2.867 Q-error, 1.187 regret) → ISOMER (1.664, 1.063) → the learned completion under projection (1.401, 1.029) → the Router (1.375, 1.029, 92.9% join-optimal), approaching

Table 12: Generator-driven optimizer-decision proxy. Q-Error and regret are geometric means; regret is true proxy cost of the plan selected from estimated statistics divided by the true optimal proxy cost (1.0 is optimal). No query runtime is measured.

Method	Sel. QE	Join Regret	Join Opt.	Fresh Match	Risk Resolved
Stale	2.867	1.187	79.0%	79.0%	0.0%
ISOMER	1.664	1.063	88.8%	88.8%	54.3%
OASIS-noProj	1.565	1.042	90.9%	90.9%	66.1%
OASIS	1.401	1.029	92.8%	92.8%	74.5%
Soft	1.362	1.025	93.2%	93.2%	75.3%
Router	1.375	1.028	92.9%	92.9%	74.0%
Fresh	1.000	1.000	100.0%	100.0%	100.0%

fresh (1.000). The Router improves on hard projection while never selecting a higher-residual candidate. Robustness along the remaining axes is consistent with the mechanism: the feedback-budget sweep (Figure 4) is the rank curve, the per-intensity q columns span drift severity, leave-one-family-out validation probes drift family (the prior trained on five families and tested on a held-out sixth beats the classical priors and ties ISOMER), and under 10% multiplicative feedback noise the Router degrades gracefully—selectivity Q-error rises only from 1.375 to 1.418 and join-optimal choices from 92.9% to 92.2%, still far below ISOMER (1.758) and stale (2.867). The failure modes are reported, not hidden: the raw prior is harmful in FactorJoin and on the NASA trace, which is why the projection and residual gating are part of the method.

6.3. Overhead and Deployment

Per-correction OASIS inference averages 1.06 ms (tensorization plus one forward pass); the PostgreSQL-style format conversion adds 0.066–0.073 ms, for a combined $\approx$ 1.13 ms per column. Feedback collection adds bounded per-conjunct bookkeeping, emitting one observation tuple per conjunct at query completion. A correction therefore costs one forward pass plus the projection and avoids the full-table `ANALYZE` scan a refresh would incur—the intended use case is maintaining optimizer-facing statistics *between* scheduled refreshes, not replacing full refreshes after major schema changes or bulk shifts.

7. Related Work

Feedback-driven statistics and the completion question. Cost-based optimization and selectivity estimation rely on compact statistics built from scans [1, 2, 3, 4], with streaming summaries such as KLL sketches providing approximate quantiles [25] and query feedback long used to adapt estimates [26]. Self-tuning histograms, STHoles, SASH, and ISOMER refine statistics from observed results [12, 13, 14, 15], and execution-feedback frameworks repair optimizer state over time [27]. OASIS shares this statistics-facing target but takes a different position: ISOMER's maximum-entropy projection is a *completion* of the feedback-underdetermined marginal, and we replace the optimizer-agnostic completion with one trained against downstream optimizer error, bounded by a rank analysis of when any completion can help. Feedback consistency and maximum-entropy estimation are long established; our contribution is, to our knowledge, the first treatment of feedback-driven statistics repair as an underdetermined inverse problem with an explicit nullspace characterization.

Learned cardinality estimation. Learned estimators such as NeuroCard, Naru, DeepDB, FACE, MSCN, FLAT, and Iris predict cardinalities from learned density or representation models [28, 29, 30, 31, 17, 32, 33], and benchmark and deployment studies document both their promise and the difficulty of robustness across workloads and drift [34, 35, 36, 10, 19, 20, 18, 37, 38, 21]. This line remains active in this journal: recent work learns dynamic sample selectors for cardinality estimation [39] and recursive-network models of complex predicates [40]. These systems output point estimates or estimator state; OASIS instead outputs corrected marginal *statistics* that an existing CBO consumes unchanged, and it trains the prior on the optimizer's error rather than on cardinality reconstruction, the same distinction the ablation draws internally.

Composition, joins, and plan-level learning. Copula and factorized-join estimators such as FactorJoin [24] combine one-dimensional marginals with a dependence or join-key model; our composition-family and FactorJoin experiments embed OASIS as the marginal source in exactly these estimators and show feedback consistency is a prerequisite for safe downstream use. Plan-level and robust optimization systems—mid-query re-optimization, progressive optimization, LEO, Neo, Bao, and Flow-Loss [41, 42, 43, 44, 45, 46, 47, 11]—

adjust plan choices or train plan-relevant estimates; OASIS operates below them by correcting the statistics object itself, and the two are complementary.

8. Limitations

The most important boundary is external validity: OASIS is not validated end-to-end on a real DBMS query workload with its own statistics-refresh history. This boundary is partly structural. To our knowledge no public dataset of production statistics-refresh histories exists, and the self-tuning-histogram line we build on and compare against—STHoles, ISOMER, and QuickSel-H—was likewise established on synthetic or controlled drift rather than on such histories. We therefore adopt the evaluation regime of the prior art and climb a deliberate ladder of increasing realism on top of it: controlled drift families; benchmark-inspired insert/update/delete DML traces; injection into a *real* PostgreSQL optimizer through `pg_statistic`, scored on `EXPLAIN` estimates and plan shapes; a cross-schema TPC-H check that reuses the *same* pretrained prior with no retraining; and one real public production trace (NASA HTTP), which we treat precisely as an append-only event table rather than a database query log. Each rung is a genuine step beyond the baselines' original validation, but none is a production refresh history, and validating the same policy end-to-end on one remains the primary future direction.

The mechanism is also intrinsically bounded: once feedback pins the marginal the learned completion provably cannot beat the projection and ties ISOMER under the same finite representation and projection operator, so the gains are concentrated where feedback leaves a nullspace—away from densely constrained regions—and we claim them only there. OASIS does not infer multi-column dependence from single-column feedback; where joint-cardinality error is dominated by cross-column correlation an external dependence model remains necessary. The TPC-H check shows only that calibrated statistics reproduce fresh-statistics execution time on one schema and six queries; runtime in general depends on executor behavior, caching, physical design, and cost-model calibration, and a stale mis-estimate can yield a coincidentally faster plan.

Finally, the Router gates on feedback residual, which is a proxy for estimate quality rather than for plan-shape risk: it selects ISOMER on the dense planner configurations, where ISOMER, OASIS, and the Router all sit near the same $\approx 1\%$ new-deviation rate (Table 9), so it does not regress

there; but the residual would not by itself flag a plan-shape regression if one arose. A plan-shape-aware gate is left to future work.

9. Conclusion

OASIS repairs stale single-column optimizer statistics from query feedback without rescanning tables or modifying the optimizer. Its organizing idea is that feedback repair is an underdetermined inverse problem: feedback pins the marginal only on the span of the observed predicates and leaves a nullspace that some completion must fill. The classical methods fill it with maximum entropy; OASIS fills it with a completion trained on a composite optimizer-facing objective (future-predicate error, join regret, feedback consistency, and a reconstruction regularizer), and a rank analysis describes where this can help. On held-out single-column drift the learned completion cuts future-predicate Q-error by 12.8% over ISOMER and beats STHoles and QuickSel-H, with the gain rising to 21% on predicates farthest from feedback, present from the sparsest feedback (+4.3% at K=2) and persisting across budgets, while, under the same finite representation and projection operator, it provably ties ISOMER only where dense feedback saturates the constraints and pins the marginal. An ablation isolates the gain to the objective and shows in-loop projection is inessential.

The same feedback projection and a residual-gated router make the imperfect completion safe to consume: OASIS improves a six-estimator composition family by 20%–36%, rescues a FactorJoin kernel where the raw prior is harmful, cuts PostgreSQL new plan deviations to 1.1%, and on a TPC-H sanity check tracks fresh-statistics behavior—accuracy and plan shape, with runtime within the reported distribution—without the table scan. OASIS is best understood as a reusable, feedback-calibrated statistics layer that learns the optimizer-relevant completion of the feedback nullspace and falls back to the maximum-entropy default where the nullspace closes.

Declarations

Data availability. The drift generators, evaluation harness, and the scripts that reproduce every table and figure in this paper (including the random seeds) are publicly available at `https://github.com/M1zukiqwq/OASIS-OpenSource`; the NASA Kennedy Space Center HTTP trace used in Section 6.1.4 is publicly available from its original source [22].

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The authors declare no competing interests.

CRediT author contributions. Qichu Tian: Conceptualization, Methodology, Software, Investigation, Data curation, Validation, Visualization, Writing – original draft. **Heng Chen**: Supervision, Conceptualization, Writing – review & editing, Project administration.

Generative AI disclosure. During the preparation of this manuscript the authors used a large language model (Anthropic Claude) to assist with language editing, manuscript restructuring, and the drafting of figure- and table-generating code. No AI tool was used to design the study, to generate or analyze experimental results, or to produce scientific claims. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, T. G. Price, Access path selection in a relational database management system, in: Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, ACM, 1979, pp. 23–34. doi:10.1145/582095.582099.

[2] S. Chaudhuri, An overview of query optimization in relational systems, in: Proceedings of the 17th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, ACM Press, 1998, pp. 34–43. doi:10.1145/275487.275492.

[3] Y. E. Ioannidis, V. Poosala, Universality of serial histograms, in: Proceedings of the 19th VLDB Conference, 1993, pp. 256–267.

[4] V. Poosala, Y. E. Ioannidis, Selectivity estimation without the attribute value independence assumption, in: Proceedings of the 23rd VLDB Conference, 1997, pp. 486–495.

[5] PostgreSQL Global Development Group, PostgreSQL documentation: Planner statistics, https://www.postgresql.org/docs/current/planner-stats.html, accessed 2026-06-01 (2026).

[6] PostgreSQL Global Development Group, PostgreSQL documentation: Row estimation examples, https://www.postgresql.org/docs/current/row-estimation-examples.html, accessed 2026-06-01 (2026).

[7] PostgreSQL Global Development Group, PostgreSQL documentation: ANALYZE, https://www.postgresql.org/docs/current/sql-analyze.html, accessed 2026-06-01 (2026).

[8] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proceedings of the VLDB Endowment 9 (3) (2015) 204–215.

[9] G. Moerkotte, T. Neumann, G. Steidl, Preventing bad plans by bounding the impact of cardinality estimation errors, Proceedings of the VLDB Endowment 2 (1) (2009) 982–993.

[10] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cai, Y. Zhang, G. Li, Cardinality estimation in DBMS: A comprehensive benchmark evaluation, Proceedings of the VLDB Endowment 15 (4) (2021) 752–765.

[11] K. Lee, A. Dutt, V. R. Narasayya, S. Chaudhuri, Analyzing the impact of cardinality estimation on execution plans in microsoft SQL server, Proceedings of the VLDB Endowment 16 (11) (2023) 2871–2883. doi:10.14778/3611479.3611494.

[12] A. Aboulnaga, S. Chaudhuri, Self-tuning histograms: Building histograms without looking at data, in: Proceedings of SIGMOD, 1999, pp. 181–192.

[13] N. Bruno, S. Chaudhuri, L. Gravano, STHoles: A multidimensional workload-aware histogram, in: Proceedings of SIGMOD, 2001, pp. 211–222.

[14] L. Lim, M. Wang, J. S. Vitter, SASH: A self-adaptive histogram set for dynamically changing workloads, in: Proceedings of the 29th VLDB Conference, 2003, pp. 369–380. doi:10.1016/B978-012722442-8/50040-9.

[15] V. Markl, N. Megiddo, M. Kutsch, T. M. Tran, P. Lang, V. Raman, Consistent selectivity estimation via maximum entropy, The VLDB Journal 16 (2) (2007) 169–188.

[16] Y. Park, S. Zhong, B. Mozafari, QuickSel: Quick selectivity learning with mixture models, in: Proceedings of SIGMOD, 2020, pp. 1017–1033.

[17] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, A. Kemper, Learned cardinalities: Estimating correlated joins with deep learning, in: CIDR, 2019.

[18] P. Li, W. Wei, R. Zhu, B. Ding, J. Zhou, H. Lu, ALECE: An attention-based learned cardinality estimator for SPJ queries on dynamic workloads, Proceedings of the VLDB Endowment 17 (2) (2023) 197–210. doi:10.14778/3626292.3626302.

[19] B. Li, Y. Lu, S. Kandula, Warper: Efficiently adapting learned cardinality estimators to data and workload drifts, in: Proceedings of SIGMOD, ACM, 2022, pp. 2146–2160. doi:10.1145/3514221.3517914.

[20] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, M. Alizadeh, Robust query driven cardinality estimation under changing workloads, Proceedings of the VLDB Endowment 16 (6) (2023) 1520–1533. doi:10.14778/3583140.3583164.

[21] Y. Han, H. Wang, L. Chen, Y. Dong, X. Chen, B. Yu, C. Yang, W. Qian, ByteCard: Enhancing ByteDance's data warehouse with learned cardinality estimation, in: Companion of the 2024 International Conference on Management of Data, ACM, 2024, pp. 41–54. doi:10.1145/3626246.3653376.

[22] Internet Traffic Archive, NASA-HTTP: Two months of HTTP logs from the Kennedy Space Center WWW server, https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html, accessed 2026-06-01 (1995).

[23] M. F. Arlitt, C. L. Williamson, Web server workload characterization: The search for invariants, in: Proceedings of the 1996 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 1996, pp. 126–137. doi:10.1145/233013.233034.

[24] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, S. Madden, FactorJoin: A new cardinality estimation framework for join queries, Proceedings of the ACM on Management of Data (SIGMOD) 1 (1) (2023) 1–27.

[25] N. Ivkin, J. Nelson, H. Yu, V. Braverman, KLL: Approximately optimal streaming quantile estimation, Proceedings of the ACM on Management of Data 2 (1) (2019) 1–23.

[26] C. M. Chen, N. Roussopoulos, Adaptive selectivity estimation using query feedback, in: Proceedings of the 24th ACM SIGMOD International Conference on Management of Data, ACM Press, 1994, pp. 161–172. doi:10.1145/191839.191874.

[27] S. Chaudhuri, V. R. Narasayya, R. Ramamurthy, A pay-as-you-go framework for query execution feedback, Proceedings of the VLDB Endowment 1 (1) (2008) 1141–1152. doi:10.14778/1453856.1453977.

[28] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, I. Stoica, NeuroCard: One cardinality estimator for all tables, Proceedings of the VLDB Endowment 14 (1) (2020) 61–73.

[29] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, I. Stoica, Deep unsupervised cardinality estimation, Proceedings of the VLDB Endowment 13 (3) (2019) 279–292.

[30] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, C. Binnig, DeepDB: Learn from data, not from queries!, Proceedings of the VLDB Endowment 13 (7) (2020) 992–1005.

[31] J. Wang, C. Chai, J. Liu, G. Li, FACE: A normalizing flow based cardinality estimator, Proceedings of the VLDB Endowment 15 (1) (2021) 72–84. doi:10.14778/3485450.3485458.

[32] R. Zhu, Z. Wu, Y. Han, K. Zeng, A. Pfadler, Z. Qian, J. Zhou, B. Cui, FLAT: Fast, lightweight and accurate method for cardinality estimation, Proceedings of the VLDB Endowment 14 (9) (2021) 1489–1502. doi:10.14778/3461535.3461539.

[33] Y. Lu, S. Kandula, A. C. König, S. Chaudhuri, Pre-training summarization models of structured datasets for cardinality estimation, Proceedings of the VLDB Endowment 15 (3) (2022) 414–426. doi:10.14778/3494124.3494127.

[34] X. Wang, C. Qu, W. Wu, J. Wang, Q. Zhou, Are we ready for learned cardinality estimation?, Proceedings of the VLDB Endowment 14 (9) (2021) 1640–1654. doi:10.14778/3461535.3461552.

[35] J. Sun, J. Zhang, Z. Sun, G. Li, N. Tang, Learned cardinality estimation: A design space exploration and a comparative evaluation, Proceedings of the VLDB Endowment 15 (1) (2021) 85–97. doi:10.14778/3485450.3485459.

[36] K. Kim, J. Jung, I. Seo, W.-S. Han, K. Choi, J. Chong, Learned cardinality estimation: An in-depth study, in: Proceedings of SIGMOD, ACM, 2022, pp. 1214–1227. doi:10.1145/3514221.3526154.

[37] Z. Wang, Q. Zeng, N. Wang, H. Lu, Y. Zhang, CEDA: Learned cardinality estimation with domain adaptation, Proceedings of the VLDB Endowment 16 (12) (2023) 3934–3937. doi:10.14778/3611540.3611589.

[38] R.-A. Wang, Z. Zou, Z. Jing, Cardinality estimation via learned dynamic sample selection, Information Systems 117 (2023) 102252. doi:10.1016/j.is.2023.102252.

[39] R.-A. Wang, Z. Zou, Z. Jing, Cardinality estimation via learned dynamic sample selection, Information Systems 117 (2023) 102252. doi:10.1016/j.is.2023.102252.

[40] Z. Wang, H. Duan, Y. Cheng, G. Min, Learning complex predicates for cardinality estimation using recursive neural networks, Information Systems 124 (2024) 102402. doi:10.1016/j.is.2024.102402.

[41] N. Kabra, D. J. DeWitt, Efficient mid-query re-optimization of sub-optimal query execution plans, in: Proceedings of SIGMOD, 1998, pp. 106–117.

[42] V. Markl, V. Raman, D. E. Simmen, G. M. Lohman, H. Pirahesh, Robust query processing through progressive optimization, in: Proceedings of SIGMOD, ACM, 2004, pp. 659–670. doi:10.1145/1007568.1007642.

[43] B. Babcock, S. Chaudhuri, Towards a robust query optimizer: A principled and practical approach, in: Proceedings of SIGMOD, ACM, 2005, pp. 119–130. doi:10.1145/1066157.1066172.

[44] M. Stillger, G. Lohman, V. Markl, M. Kandil, LEO – DB2's learning optimizer, in: Proceedings of the 27th VLDB Conference, 2001, pp. 19–28.

[45] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, N. Tatbul, Neo: A learned query optimizer, Proceedings of the VLDB Endowment 12 (11) (2019) 1705–1718.

[46] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, in: Proceedings of SIGMOD, 2022, pp. 1275–1288.

[47] P. Negi, R. Marcus, A. Kipf, H. Mao, N. Tatbul, T. Kraska, M. Alizadeh, Flow-loss: Learning cardinality estimates that matter, Proceedings of the VLDB Endowment 14 (11) (2021) 2019–2032. doi:10.14778/3476249.3476259.